GSD RESEARCH MISSION PACKAGE

NeHe OpenGL Intelligence Pack

48 Lessons · All Examples · Upstream Integration

skill-creator Code, Documentation & Working Examples

Vision → Research → Mission

Full Pipeline Execution

Date: March 27, 2026 | Status: Ready for GSD Orchestrator

Activation Profile: Fleet (17 roles) | Estimated: 8–12 context windows across 3 sessions

Source: nehe.gamedev.net | Archive: github.com/gamedev-net/nehe-opengl

Contents

1 NeHe OpenGL Intelligence Pack — Vision Guide **4**
1.1 Vision . . . 4
1.2 Problem Statement . . . 4
1.3 Core Concept . . . 5
1.3.1 Rendering Knowledge Graph — Domain Map . . . 6
1.3.2 Module Architecture . . . 6
1.4 Research Modules . . . 6
1.4.1 Module 1 — Taxonomy & Skill Mapping . . . 7
1.4.2 Module 2 — Foundation Track (Lessons 01–12) . . . 7
1.4.3 Module 3 — Intermediate Track (Lessons 13–24) . . . 7
1.4.4 Module 4 — Advanced Track (Lessons 25–36) . . . 7
1.4.5 Module 5 — Expert Track (Lessons 37–48) . . . 7
1.4.6 Module 6 — Upstream Intelligence Integration . . . 7
1.5 Chipset Configuration . . . 8
1.6 Scope Boundaries . . . 8
1.6.1 In Scope (v1.0) . . . 8
1.6.2 Out of Scope . . . 8
1.7 Success Criteria . . . 9
1.8 Relationship to Other Documents . . . 10
1.9 The Through-Line . . . 10

2 NeHe OpenGL — Research Reference **11**
2.1 How to Use This Document . . . 11
2.2 Complete Lesson Taxonomy (All 48 Lessons) . . . 11
2.3 Module 2 Reference — Foundation Track Implementation Patterns . . . 20
2.3.1 Cross-Platform Window Setup (Lesson 01 — Modern Python) . . . 20
2.3.2 Texture Loading Pattern (Lesson 06) . . . 21
2.4 Module 3 Reference — Intermediate Track Key Patterns . . . 21
2.4.1 Particle Engine (Lesson 19) . . . 21
2.4.2 Bump Mapping / Multitexturing (Lesson 22) . . . 21
2.5 Module 4 Reference — Advanced Track Key Patterns . . . 21
2.5.1 Stencil Buffer Pattern (Lessons 26–27) . . . 21
2.5.2 Height Map Terrain (Lesson 34) . . . 21
2.6 Module 5 Reference — Expert Track Key Patterns . . . 21
2.6.1 Vertex Buffer Objects (Lesson 45) . . . 22
2.6.2 ArcBall Rotation (Lesson 48) . . . 22
2.6.3 FreeType Fonts (Lesson 43) . . . 22
2.7 Module 6 Reference — Upstream Intelligence Integration . . . 22
2.7.1 DACP Bundle Schema for OpenGL Lessons . . . 22
2.7.2 Deprecation Map — Legacy to Modern OpenGL . . . 23
2.8 Safety and Sensitivity Considerations . . . 24
2.9 Source Bibliography . . . 24
2.9.1 Primary Sources . . . 24
2.9.2 Supporting Technical References . . . 24

3 Milestone Specification **25**
3.1 Mission Objective . . . 25
3.2 Architecture Overview . . . 25
3.3 System Layers . . . 25
3.4 Deliverables . . . 26
3.5 Component Breakdown . . . 27
3.6 Model Assignment Rationale . . . 28

4 Wave Execution Plan **28**
4.1 Wave Summary . . . 29
4.2 Wave 0 — Foundation . . . 29
4.3 Wave 1 — Parallel Tracks (A, B, C) . . . 29
4.3.1 Track A — Taxonomy and Intelligence . . . 30
4.3.2 Track B — Foundation & Intermediate Code (L01–L24) . . . 30
4.3.3 Track C — Advanced & Expert Code (L25–L48) . . . 30
4.4 Cache Optimization Strategy . . . 31
4.5 Token Budget . . . 31
4.6 Dependency Graph . . . 31

5 Test Plan **31**
5.1 Test Categories . . . 32
5.2 Safety-Critical Tests (BLOCK delivery if fail) . . . 32
5.3 Core Functionality Tests . . . 32
5.4 Verification Matrix . . . 33

6 Mission Crew Manifest **34**

7 Execution Summary **35**

STAGE 1 — VISION DOCUMENT

1 NeHe OpenGL Intelligence Pack — Vision Guide

Date: March 27, 2026

Status: Cold-Start Mission / Research Compiled

Depends on: gsd-skill-creator (dev, v1.49+), gsd-upstream-intelligence-pack-v1_43.md, gsd-silicon-layer-spec.md

Context: Adding the complete NeHe OpenGL tutorial corpus (48 lessons, 43 platform ports) to skill-creator as upstream intelligence, working code examples, and documentation — the canonical 3D graphics pedagogy of the late 1990s to 2010s, preserved and modernized.

1.1 Vision

There are corpora that defined entire generations of programmers. Jeff Molofee's NeHe Productions — Neon Helium — was that corpus for real-time 3D graphics. Between 1997 and 2012, the NeHe tutorials taught OpenGL to hundreds of thousands of developers: the ones who built the games, the simulations, the visualizers, the CAD tools. Every lesson was a handshake between a beginner and a coordinate system, between a curious programmer and a rendering pipeline.

The 48 lessons form a complete developmental arc: from the first blank window (Lesson 01) through texture mapping, lighting, blending, particle systems, shadow volumes, collision detection, and finally ArcBall rotation (Lesson 48). Each lesson builds on the last — a curriculum so well-sequenced that it became the de facto OpenGL textbook for a decade. The GitHub archive preserves source code across 43 platform ports: C++, Python, Java, Delphi, Perl, Ruby, Scheme, Fortran, Assembly, and more.

The GSD ecosystem is about architectural intelligence: building systems that *know* their domain deeply enough to adapt, explain, and generate within it. The NeHe corpus is exactly the kind of structured, layered knowledge that skill-creator should hold. Not as static reference, but as *executable intelligence* — a taxonomy of 3D concepts mapped to skills, with working modern implementations ready for agent deployment. When a future skill-creator session encounters a request involving vertex buffers or shadow stencils or particle systems, it should be able to reach into this corpus and surface the right lesson, the right pattern, the right code.

The modernization mandate is real: NeHe's original code runs on Windows with legacy Win32 API and deprecated glaux. The working examples for this mission target Python with ModernGL or PyOpenGL + GLFW — cross-platform, pip-installable, runnable on any developer machine without a Visual C++ IDE from 2002.

1.2 Problem Statement

1. **No structured 3D graphics knowledge in skill-creator.** The current upstream intelligence architecture monitors Anthropic channels and skill speci-

fications, but holds no domain knowledge about computer graphics, rendering pipelines, or mathematical foundations of 3D space. Requests involving OpenGL, shaders, or 3D math require the orchestrator to synthesize knowledge from scratch.

2. **Legacy code is non-runnable without archaeology.** The GitHub archive's C++ examples require Win32, glaux, and legacy build systems. Python ports exist but are fragmented across contributors with inconsistent quality. There is no single tested, modern, cross-platform implementation of all 48 lessons.
3. **No skill taxonomy maps graphics concepts to skill-creator primitives.** The NeHe lessons span categories (setup, primitives, transforms, textures, lighting, advanced effects, physics) that map naturally to skill clusters, but this mapping doesn't exist in machine-readable form. A skill that generates OpenGL code should know whether it's operating in the foundation tier or the expert tier.
4. **Upstream intelligence gap for graphics API evolution.** The transition from legacy OpenGL (fixed-function pipeline) to modern OpenGL (programmable shaders, VBOs, VAOs) is architecturally significant. Skill-creator has no model of this transition — no awareness of deprecation boundaries, no ability to flag when generated code uses obsolete patterns.
5. **Missing DACP bundles for graphics skill handoffs.** Agent-to-agent handoffs in graphics tasks should pass structured three-part bundles (human intent + lesson reference + executable code), not ambiguous markdown. No DACP schema exists for OpenGL lesson delivery.

1.3 Core Concept

Model: Lesson as Skill Node, Corpus as Directed Graph

Each NeHe lesson is a skill node with defined prerequisites, concepts introduced, and successors. The 48 lessons form a directed acyclic graph (DAG) — the rendering knowledge graph — that can be traversed by skill-creator to answer graphics questions, generate code at the right complexity level, or build multi-lesson learning sequences.

1.3.1 Rendering Knowledge Graph — Domain Map

```
NEHE OPENGL CORPUS --- SKILL GRAPH OVERVIEW
TIER 0: FOUNDATION (L01-L05) L01 Window Setup ──► L02 Primitives ──► L03
Color ──► L04 Rotation ──► L05 3D Shapes
TIER 1: TEXTURE  LIGHT (L06-L12) L06 Texture Mapping ──► L07
Filters+Lighting+KB ──► L08 Blending L05 ──► L09 Moving Bitmaps ──► L10
3D World Loading L10 ──► L11 Flag/Wave Effect ──► L12 Display Lists
TIER 2: FONTS  EFFECTS (L13-L20) L12 ──► L13 Bitmap Fonts ──► L14 Outline
Fonts ──► L15 Texture Mapped Fonts L08 ──► L16 Fog ──► L17 2D Texture Font
──► L18 Quadratics L08 ──► L19 Particle Engine ──► L20 Masking
TIER 3: ADVANCED RENDERING (L21-L28) L07 ──► L21 Lines+Antialiasing+Sound
──► L22 Bump+Multitex+Ext L06 ──► L23 Sphere Mapping ──► L24
Tokens+Scissor+TGA L25 Morphing+OBJ Load ──► L26 Stencil+Reflections ──►
L27 Shadow Volumes L28 Glow+Motion Blur
TIER 4: SIMULATION  WORLDS (L29-L36) L29 Blitter+RAW+ASM ──► L30 BSP
Collision ──► L31 Model Loading L32 Picking+Fogging+Multitex ──► L33 TGA
Compression L34 Height Maps ──► L35 AVI Playback ──► L36 Radial Blur+RTT
TIER 5: EXPERT (L37-L48) L37 Cel-Shading ──► L38 Compressed Textures ──►
L39 Physics Intro L39 ──► L40 Rope Physics ──► L41 ODE Virtual Worlds
L42 Multiple Viewports ──► L43 FreeType Fonts L44 3D Lens Flare+Occlusion
──► L45 Vertex Buffer Objects (VBO) L46 Fullscreen FSAA ──► L47 CG Vertex
Shader ──► L48 ArcBall Rotation
CROSS-CUTTING: Math Foundation ──────────────────────────────────► All
tiers DACP Bundle Schema ──────────────────────────────────► All tiers
Modern API Wrapper (PyOpenGL/ModernGL) ──────────────────► All tiers
```

1.3.2 Module Architecture

```
MODULE DECOMPOSITION (6 modules, Fleet activation)
MODULE 1: Taxonomy  Skill Mapping - Lesson-to-skill YAML taxonomy (48
entries) - Concept DAG in JSON-LD - DACP bundle schemas per lesson tier
- skill-creator YAML for "opengl-lesson" skill type
MODULE 2: Foundation Track (L01-L12) - Modern Python implementations
(PyOpenGL + GLFW) - Tested, cross-platform, pip-runnable - Inline
documentation referencing NeHe concept
MODULE 3: Intermediate Track (L13-L24) - Font rendering via
FreeType/Pygame - Effects: fog, particles, masking, bump mapping -
Multi-texture and extension handling
MODULE 4: Advanced Track (L25-L36) - OBJ loading, stencil buffer, shadows
- BSP collision, height maps, render-to-texture - TGA loading, AVI
playback abstraction
MODULE 5: Expert Track (L37-L48) - Cel-shading, physics simulations
(L39-L41) - VBOs, FSAA, CG shader translation - FreeType, ArcBall rotation
math
MODULE 6: Upstream Intelligence Integration - opengl-intelligence.yaml for
skill-creator - DACP bundle format for graphics handoffs - Deprecation
map: legacy OpenGL → modern OpenGL - Documentation indexer for lesson
lookup
```

1.4 Research Modules

1.4.1 Module 1 — Taxonomy & Skill Mapping

Produces the machine-readable knowledge graph that skill-creator uses to reason about 3D graphics requests. Output is a YAML taxonomy file, a JSON-LD concept DAG, and a skill definition that registers the "opengl-lesson" skill type. This module is the intelligence backbone — all other modules are data that flows through it.

1.4.2 Module 2 — Foundation Track (Lessons 01–12)

Implements and tests the twelve foundation lessons in modern Python. These cover: OpenGL window setup, drawing triangles and quads, adding color, rotation, 3D shapes, texture mapping, lighting and keyboard input, blending, moving bitmaps in 3D, loading a 3D world, wave/flag effect, and display lists. Every implementation uses PyOpenGL + GLFW (cross-platform) or ModernGL where the lesson maps cleanly to a modern shader pattern. Tests verify visual output via framebuffer comparison.

1.4.3 Module 3 — Intermediate Track (Lessons 13–24)

Implements lessons covering bitmap fonts, outline fonts, texture-mapped fonts, fog effects, 2D texture fonts, quadratics (spheres, cylinders, discs), particle engines, texture masking, lines and antialiasing with timing, bump mapping and multitexturing, sphere mapping, and token/extension detection with scissor testing. The font lessons require FreeType integration. Particle engine (L19) is particularly important as a skill pattern for procedural geometry generation.

1.4.4 Module 4 — Advanced Track (Lessons 25–36)

Implements morphing between OBJ meshes, stencil buffer reflections and clipping, shadow volumes, glow and motion blur effects, BSP tree collision detection, model loading, picking with fog and multitexturing, TGA loading (compressed and uncompressed), height map landscape rendering, AVI playback abstraction (cross-platform video texture), and radial blur with render-to-texture. The BSP and shadow volume lessons are the most architecturally complex and require careful dependency management.

1.4.5 Module 5 — Expert Track (Lessons 37–48)

Implements cel-shading (toon shader), compressed texture loading (DXT), introduction to physical simulations (spring-mass systems), rope physics (Verlet integration), virtual worlds with ODE physics, multiple viewport rendering, FreeType font rendering in OpenGL, 3D lens flare with occlusion testing, vertex buffer objects (the modern GPU data path), fullscreen antialiasing, CG vertex shader translation (to GLSL), and ArcBall rotation (the interaction pattern for 3D viewport navigation). This module requires the most adaptation from legacy to modern API.

1.4.6 Module 6 — Upstream Intelligence Integration

Produces the artifact that makes all other modules useful inside skill-creator: the intelligence index that agents can query. Includes: the YAML taxonomy for skill registration, DACP three-part bundle schemas for graphics skill handoffs, a deprecation map that flags legacy OpenGL patterns and suggests modern equivalents, and a documentation index that agents can search by concept keyword, lesson number, or API function name.

1.5 Chipset Configuration

```
nehe-intelligence-pack.chipset.yaml  Fleet activation: 17 roles, 6 modules
agnus: role: Orchestration / DMA / Context Management model:
claude-opus-4-6 primary_tasks : -Taxonomydesign(Module1schemaarchitecture) - Cross -
moduleDAGvalidation - DACPschemadesign - Legacy - to - modernAPIdeprecationanalysis -
Cross - modulepatterndetection(ANALYST)token_budget : 45
denise: role: Display / Output Rendering model: claude-sonnet-4-6
primary_tasks : -Pythoncodeimplementation(Modules2 - 5) - Documentationgeneration -
YAMLtaxonomycompilation - Testharnessconstruction - Indexpagegenerationtoken_budget : 45
paula: role: Audio / I/O / Anticipatory model: claude-haiku-4-5
primary_tasks : -Sharedschemasandtemplates - Filescaffolding - Lessonmetadataextraction -
Teststubgenerationtoken_budget : 10
```

1.6 Scope Boundaries

1.6.1 In Scope (v1.0)

- All 48 NeHe lessons fully documented in skill-creator taxonomy
- Working Python implementations for all 48 lessons (PyOpenGL + GLFW, cross-platform)
- Automated test harness with framebuffer validation for lessons 01–20
- Visual correctness spot-checks for lessons 21–48
- DACP bundle schema for OpenGL skill handoffs
- Deprecation map: legacy OpenGL (fixed-function) to modern OpenGL 3.3+ patterns
- skill-creator YAML registration for “opengl-lesson” skill type
- Upstream intelligence documentation index (searchable by concept, lesson, API)
- README and integration guide for adding the pack to the dev branch

1.6.2 Out of Scope

- Non-Python platform ports (C++, Java, Delphi, etc.) — defer to GitHub archive
- Vulkan or Metal equivalents
- WebGL/WebGPU translations
- Real-time audio integration (L21 sound component — stub only)
- ODE physics library full integration (L41 — interface documented, implementation deferred)
- AVI video playback (L35 — platform-specific; cross-platform stub provided)
- Full GLSL shader replacements for all fixed-function pipeline patterns (deferred to v2.0)

1.7 Success Criteria

1. All 48 lessons are represented in the skill-creator taxonomy YAML with correct metadata (tier, prerequisites, concepts, API calls).
2. All 48 Python implementations run without error on Ubuntu 24 with PyOpenGL and GLFW installed via pip.
3. Foundation lessons (01–12) produce pixel-correct output matching reference screenshots (within 2% RMSE on framebuffer comparison).
4. DACP bundle schemas validate against the three-part bundle specification (intent + data + code).
5. The deprecation map correctly flags at least 15 legacy OpenGL patterns with modern equivalents.
6. The documentation index returns the correct lesson for at least 30/30 spot-check concept queries.
7. The skill-creator YAML installs cleanly via the skill registration process on the dev branch.
8. All test harness tests pass in a fresh Ubuntu 24 environment (no undeclared dependencies).
9. Expert-tier lessons (37–48) include inline documentation explaining the mathematical concept, not just the code.
10. The ArcBall rotation implementation (L48) passes quaternion unit tests.
11. The VBO implementation (L45) uses only OpenGL 3.3 core profile features (no deprecated fixed-function calls).
12. The complete pack is deliverable as a single directory that can be dropped into the skill-creator dev branch.

1.8 Relationship to Other Documents

Document	Relationship	Notes
gsd-upstream-intelligence-pack-v1_43.md	Parent architecture	This pack extends the upstream intelligence layer into a new domain (3D graphics) using the Watch/Classify/Trace/Adapt/Brief cycle
gsd-skill-creator-analysis.md	Implementation host	The YAML taxonomy and skill definition integrate directly into skill-creator's six-step observe-detect-suggest loop
gsd-silicon-layer-spec.md	Execution model	Code examples follow the silicon layer's sandboxed execution patterns
gsd-instruction-set-architecture-vision.md	ISA mapping	OpenGL API calls can be modeled as a domain-specific instruction set within the GSD ISA framework
the-space-between.pdf	Mathematical spine	L-Systems, vector calculus, and complex plane concepts from the textbook underlie lessons 11, 25, 39–41

1.9 The Through-Line

The NeHe tutorials are not just a programming curriculum. They are a record of how human beings learned to build worlds in silicon — lesson by lesson, coordinate system by coordinate system, one glVertex call at a time. Jeff Molofee wrote them between 1997 and 2012, giving them away freely, and they became the substrate on which a generation of 3D programmers grew. The archive at `nehe.gamedev.net` is quiet now, but its lesson count — 48 — is almost exactly the number of NeHe's own life chapters worth remembering.

The GSD ecosystem is built on the Amiga Principle: remarkable outcomes through architectural elegance and specialized execution paths. The Amiga 500 — 7 MHz, 512K RAM — rendered animations that stunned engineers working with machines ten times more powerful, because its specialized chips (Agnus, Denise, Paula) each did one thing with surgical precision. The NeHe corpus teaches the same lesson in a different register: a single glTexCoord2f call, composed with a glVertex3f in a display list, inside a carefully managed state machine, produces effects that seem to exceed what the underlying math should allow.

Both are about the spaces between. The magic of a textured cube isn't in any individual vertex — it's in the mapping between texture coordinate and geometry. The magic of a particle system isn't in any individual particle — it's in the emergent field they form. This is the intelligence this pack adds to skill-creator: not a lookup table, but a *compositional grammar* of 3D space.

STAGE 2 — RESEARCH REFERENCE

2 NeHe OpenGL — Research Reference

2.1 How to Use This Document

Stage 2 provides the research findings that agents use to implement each module. Each subsection corresponds to a mission module. The lesson taxonomy (Section 2.2) is the primary data source for Module 1. Code patterns (Sections 2.3–2.7) inform the Python implementations. Section 2.8 covers the deprecation landscape. Section 2.9 is the source bibliography.

Key source repositories and references:

- **Primary archive:** `https://github.com/gamedev-net/nehe-opengl` (MIT license, 43 platform ports)
- **Python port reference:** `/python/` directory in the archive (PyOpenGL + GLUT baseline)
- **Linux/SDL port reference:** `/linuxsdl/` directory (clean C, cross-platform)
- **Documentation PDF:** NeHe's OpenGL Tutorials (Andreas Lagotzki, 2004, 482 pages)
- **Original site:** `http://nehe.gamedev.net` (1997–2012, Jeff Molofee "NeHe")

2.2 Complete Lesson Taxonomy (All 48 Lessons)

Lessor	Title	Tier	Comple
Key Con-cepts			
01	Setting Up An OpenGL Window	Foundation	Low
Context cre-ation, ren-der-ing loop, GLFW			
02	Your First Polygon	Foundation	Low
glVertex, GL_TRIANGLES, GL_QUADS			
03	Adding Color	Foundation	Low

Lessor	Title	Tier	Comple
Key Con-cepts			
glColor3f, smooth shad-ing, flat shad-ing			
04	Rotation	Foundation	Low
glRotatef, ani-ma-tion loop, delta time			
05	3D Shapes	Foundation	Low
glTranslatef, 3D primi-tives, depth buffer			
06	Texture Mapping	Foundation	Medium
glGenTextures, BMP load-ing, UV coor-di-nates			
07	Filters, Lighting, Keyboard	Foundation	Medium
GL_LIGHT0, glMa-teri-alfv, key han-dling			
08	Blending	Foundation	Medium

Lesson	Title	Tier	Complexity	Key Concepts
				glBlendFunc, GL_SRC_ALPHA, transparency
09	Moving Bitmaps In 3D Space	Foundation	Medium	Billboard sprites, depth testing
10	Loading A 3D World	Foundation	Medium	Text-based world format, sectors
11	Flag Effect (Waving Texture)	Foundation	Medium	Sine-based vertex displacement, wave sim
12	Display Lists	Foundation	Medium	glNewList, glCallList, GL_COMPILE
13	Bitmap Fonts	Intermediate	Medium	wglUseFontBitmaps, glBitmap, font metrics
14	Outline Fonts	Intermediate	Medium	

Lessor	Title	Tier	Comple
Key Con-cepts			
wglUseFontOutlines, 3D ex-truded glyphs			
15	Texture Mapped Outline Fonts	Intermediat	Medium
Font on ro-tating ge-ome-try			
16	Cool Looking Fog	Intermediat	Low
glFogf, GL_EXP2, fog den-sity			
17	2D Texture Font	Intermediat	Medium
Texture atlas font, sprite sheets			
18	Quadratics	Intermediat	Medium
gluSphere, gluCylin-der, gluD-isk			
19	Particle Engine (Triangle Strips)	Intermediat	High
GL_TRIANGLE_STRIP, grav-ity, color fade			
20	Masking	Intermediat	Medium

Lessor	Title	Tier	Comple
Key Con-cepts			
Stencil-based mask-ing, logi-cal ops			
21	Lines, Antialiasing, Timing, Sound	Intermediat	High
GL_LINE_LOOP, GL_SMOOTH, timer, audio			
22	Bump Mapping & Multitexturing	Intermediat	High
GL_TEXTURE1_ARB, EMBM, ex-ten-sions			
23	Sphere Mapping	Intermediat	Medium
GL_SPHERE_MAP, envi-ron-ment map-ping			
24	Tokens, Scissor Testing, TGA	Intermediat	Medium
glScissor, TGA for-mat, ex-ten-sion to-kens			
25	Morphing & OBJ Loading	Advanced	High

Lessor	Title	Tier	Comple
Key Con-cepts			
Vertex inter-pola-tion, Wave-front OBJ			
26	Stencil Buffer Reflections	Advanced	High
glStencilFunc, pla-nar re-flec-tion			
27	Shadow Volumes	Advanced	High
Stencil shadow vol-umes, sil-hou-ette edges			
28	Glow & Motion Blur	Advanced	High
Accumulation buffer, glow mask			
29	Blitter, RAW Textures, ASM	Advanced	High
Raw tex-ture for-mat, soft-ware blit			
30	Collision Detection (BSP Trees)	Advanced	Very High

Lessor	Title	Tier	Comple
Key Con-cepts			
BSP tree con-struc-tion, AABB, ray cast			
31	Model Loading	Advanced	High
MD2 for-mat, keyframe ani-ma-tion			
32	Picking, Fog, Multitexturing	Advanced	High
glSelectBuffer, render-for-pick			
33	Loading Com-pressed/Uncompressed TGA	Advanced	Medium
TGA RLE, BGRA con-ver-sion			
34	Height Maps	Advanced	High
Heightmap sam-pling, ter-rain mesh gen			
35	Playing AVI Files In OpenGL	Advanced	High

Lessor	Title	Tier	Comple
Key Con-cepts			
Video tex-ture stream-ing, Di-rect-Show			
36	Radial Blur, Render-to-Texture	Advanced	High
FBO, radial blur shader, RTT pat-tern			
37	Cel-Shading (Toon Shading)	Expert	High
1D tex-ture lookup, sil-hou-ette pass			
38	Loading Compressed Textures	Expert	Medium
DXT1/3/5, GL_EXT_texture_compression			
39	Introduction To Physical Simulations	Expert	High
Spring-mass sys-tem, Euler inte-gra-tion			
40	Rope Physics	Expert	High

Lessor	Title	Tier	Comple
Key Con-cepts			
Verlet inte-gra-tion, con-straint solv-ing			
41	Virtual Worlds With ODE	Expert	Very High
Open Dy-nam-ics En-gine, rigid body sim			
42	Multiple Viewports	Expert	Medium
glViewport, multi-ple cam-eras			
43	FreeType Fonts In OpenGL	Expert	High
FreeType2, glyph atlas, SDF readi-ness			
44	3D Lens Flare With Occlusion	Expert	High
GL_QUERY, occlu-sion test, flare fade			
45	Vertex Buffer Objects (VBO)	Expert	High

Lessor	Title	Tier	Comple
Key Con-cepts			
glGenBuffers, GL_ARRAY_BUFFER, VAO			
46	Fullscreen AntiAliasing (FSAA)	Expert	Medium
WGL_SAMPLE_BUFFERS, multi-sam-pling			
47	CG Vertex Shader	Expert	High
Cg toolkit, GLSL trans-lation			
48	ArcBall Rotation	Expert	High
Quaternion math, track-ball inter-ac-tion			

2.3 Module 2 Reference — Foundation Track Implementation Patterns

2.3.1 Cross-Platform Window Setup (Lesson 01 — Modern Python)

The original NeHe Lesson 01 uses Win32 API (CreateWindowEx, WNDCLASS, wglCreateContext). The modern cross-platform equivalent uses GLFW via the `pyglfw` package. Key pattern:

```
import glfw from OpenGL.GL import *
def main(): if not glfw.init(): raise RuntimeError("GLFW init failed")
glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)glfw.wind
glfw.create_window(800, 600, "NeHeL01−OpenGLWindow", None, None)glfw.make_context_current(window)glClearColor(0.0, 0.
glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT)glfw.swap_buffers(window)glfw.poll_events()glfw.terminate()
```

Note on legacy vs. modern API: Lessons 01–12 use the fixed-function pipeline (glBegin/glEnd, glRotatef, glTranslatef, glColor3f). The implementations preserve this for pedagogical fidelity — the DACP skill metadata flags these as "legacy-compatible" and the deprecation map notes the modern VBO+shader equivalents.

2.3.2 Texture Loading Pattern (Lesson 06)

The BMP loading in the original uses Windows-specific bitmap functions. Modern replacement uses `Pillow` (PIL):

```
from PIL import Image import numpy as np
def load_texture(filename) : img = Image.open(filename).convert("RGB")img_data =
np.array(img, dtype = np.uint8)tex_id = glGenTextures(1)glBindTexture(GL_TEXTURE_2D, tex_id)glTexParameteri(GL_TEXTU
```

2.4 Module 3 Reference — Intermediate Track Key Patterns

2.4.1 Particle Engine (Lesson 19)

The particle engine is one of the most important intermediate patterns — it demonstrates the fundamental particle system architecture that appears in game engines, VFX tools, and GPU compute systems. Key data structure:

```
import numpy as np from dataclasses import dataclass
@dataclass class Particle: active: bool = True life: float = 1.0 fade: float =
0.025 r: float = 1.0; g: float = 0.0; b: float = 0.0 x: float = 0.0; y: float =
0.0; z: float = 0.0 xi: float = 0.0; yi: float = 0.0; zi: float = 0.0 xg: float =
0.0; yg: float = -0.8; zg: float = 0.0
MAX_PARTICLES = 1000particles = [Particle(fade = np.random.uniform(0.003, 0.025))for_inrange(MAX_PARTICLES)]
```

2.4.2 Bump Mapping / Multitexturing (Lesson 22)

Requires OpenGL extensions detection. Modern pattern uses `OpenGL.extensions`:

```
from OpenGL.extensions import hasGLExtension
if hasGLExtension('GL_ARB_multitexture') : fromOpenGL.GL.ARB.multitextureimport *
glActiveTextureARB(GL_TEXTURE0_ARB)glBindTexture(GL_TEXTURE_2D, base_tex)glActiveTextureARB(GL_TEXTURE1_A
```

2.5 Module 4 Reference — Advanced Track Key Patterns

2.5.1 Stencil Buffer Pattern (Lessons 26–27)

The stencil buffer enables reflections and shadow volumes. Critical state management:

```
Reflection pattern (Lesson 26) glEnable(GL_STENCIL_TEST)glStencilFunc(GL_ALWAYS, 1, 0xFF)glStencilOp(GL_
1onY)glPushMatrix()glScalef(1.0, −1.0, 1.0)draw_scene()glPopMatrix()glDisable(GL_STENCIL_TEST)
```

2.5.2 Height Map Terrain (Lesson 34)

```
def get_height(map_data, x, z, scale = 10) : map_data : numpyarrayfromgrayscaleimagereturnfloat(map_data[z, x])∗
scale/255.0
def build_terrain(map_data, width, depth) : vertices = []forzinrange(depth − 1) : forxinrange(width) :
fordzin[0, 1] : h = get_height(map_data, x, z + dz)vertices.extend([x − width/2, h, z + dz −
depth/2])returnnp.array(vertices, dtype = np.float32)
```

2.6 Module 5 Reference — Expert Track Key Patterns

2.6.1 Vertex Buffer Objects (Lesson 45)

VBOs are the most architecturally significant modern OpenGL pattern in the NeHe series — the transition from per-vertex CPU calls to GPU-resident data:

```
VBO + VAO pattern (Lesson 45 modernized) import numpy as np from OpenGL.GL import
*
def create_vbo(vertices, indices) : vao = glGenVertexArrays(1)vbo = glGenBuffers(1)ebo =
glGenBuffers(1)glBindVertexArray(vao)glBindBuffer(GL_ARRAY_BUFFER, vbo)glBufferData(GL_ARRAY_BUFFER, vertic
```

2.6.2 ArcBall Rotation (Lesson 48)

The ArcBall is the standard interaction model for 3D viewport navigation — maps 2D mouse movement to 3D rotation via quaternions:

```
import numpy as np from numpy.linalg import norm
def map_to_sphere(point, width, height) : """Map2Dscreenpointtounitspheresurface."""v = np.array([(2 *
point[0] − width)/width, (height − 2 * point[1])/height, 0.0])d = norm(v[: 2])v[2] = np.sqrt(max(1.0 − d *
d, 0.0))ifd < 1.0else0.0returnv/norm(v)ifnorm(v) > 0elsev
def compute_rotation(v1, v2) : """Computequaternionrotatingv1tov2onunitsphere."""perp =
np.cross(v1, v2)angle = np.dot(v1, v2)returnnp.array([perp[0], perp[1], perp[2], angle])xyzwquaternion
```

2.6.3 FreeType Fonts (Lesson 43)

```
import freetype  pip install freetype-py
def load_glyph_atlas(font_path, size = 48) : face = freetype.Face(font_path)face.set_pixel_sizes(0, size)characters =
for cinrange(128) : face.load_char(chr(c), freetype.FT_LOAD_RENDER)g = face.glyphtex =
glGenTextures(1)glBindTexture(GL_TEXTURE_2D, tex)glTexImage2D(GL_TEXTURE_2D, 0, GL_RED, g.bitmap.width, g.bitmap
'texture' : tex,' size' : (g.bitmap.width, g.bitmap.rows),' bearing' : (g.bitmap_left, g.bitmap_top),' advance' : g.advance.xreturnchara
```

2.7 Module 6 Reference — Upstream Intelligence Integration

2.7.1 DACP Bundle Schema for OpenGL Lessons

Each OpenGL skill delivery follows the DACP three-part bundle structure:

```
DACP Bundle: OpenGL Lesson Handoff  Part 1: Human Intent intent:
type: "opengl_lesson_request"lesson_id : 19tier : "intermediate"description :
"Implementparticleenginewithtrianglestrips"target_language : "python"api_profile :
"legacy_compatible"or"modern_core"
Part 2: Structured Data data: lesson_metadata : title : "ParticleEngineUsingTriangleStrips"prerequisites :
[8, 9]lessonsconcepts : ["GL_TRIANGLE_STRIP", "particle_system", "gravity"]key_apis :
["glBegin", "GL_TRIANGLE_STRIP", "glColor4f"]deprecation_flags : −pattern : "glBegin/glEnd"severity :
"info"modern_equivalent : "VBO + glDrawArrays"
Part 3: Executable Code code: entry_point : "lessons/lesson19/lesson19.py"dependencies :
["pyopengl", "pyglfw", "Pillow", "numpy"]test_command : "pytesttests/test_lesson19.py"run_command :
"pythonlessons/lesson19/lesson19.py"
```

2.7.2 Deprecation Map — Legacy to Modern OpenGL

Legacy Pattern	OpenGL Version	Modern Equivalent	Severity
glBegin/glEnd	≤2.1	VBO + glDrawArrays / glDrawElements	Core
glRotatef/glTransla	≤2.1	GLM / numpy + uniform matrix	Core
gluPerspective	≤2.1	GLM perspective() or custom	Core
glVertex/glColor/gl	≤2.1	Vertex attributes in VBO	Core
GL_LIGHTING state machine	≤2.1	Fragment shader	Core
Display lists (glNewList)	≤2.1	VAO + VBO + instancing	Core
gluSphere/gluCylin	≤2.1	Custom geometry + shader	Info
Accumulation buffer	≤2.1	Multi-pass FBO + shader	Core
GLUT (glutMainLoop)	≤2.1	GLFW / SDL2	Info
glaux library	≤1.5	Pillow / stb_image	Core
glBitmap / wglUseFontBitmaps	Windows only	FreeType2 + texture atlas	Core
CG shader (cgc)	nVidia-only	GLSL 3.30+	Core
GL_QUADS primitive	≤2.1	Two GL_TRIANGLES or GL_TRIANGLE_STRIP	Core
glTexEnvf (TexEnv combine)	≤2.1	Fragment shader texture sampling	Core
wgl* functions (context)	Windows only	GLFW / EGL / SDL2 context	Core

2.8 Safety and Sensitivity Considerations

Area	Type	Handling
Legacy code licensing	Copyright	All NeHe code is MIT licensed via the GitHub archive. All implementations must include the original copyright header and attribution to Jeff Molofee.
GPU driver variability	Technical	Tests must handle graceful fallback for missing extensions. FSAA (L46) and CG shaders (L47) require hardware-specific testing notes.
Physics simulation (L39-41)	Educationa	Physics examples are demonstrations only. No real-world safety implications. ODE library dependency noted.
Platform-specific code (L35 AVI, L29 ASM)	Compatibili	Windows-specific components (DirectShow, x86 ASM) are replaced with cross-platform stubs. Stubs are clearly documented as non-functional placeholders pending platform-specific implementation.

2.9 Source Bibliography

2.9.1 Primary Sources

- Molofee, J. ("NeHe"), *NeHe OpenGL Tutorials, Lessons 01–48*, Neon Helium Productions, 1997–2012. http://nehe.gamedev.net
- gamedev-net (GitHub org), *nehe-opengl: The complete archive of all NeHe OpenGL Lessons*, MIT License. https://github.com/gamedev-net/nehe-opengl
- Lagotzki, A., *NeHe's OpenGL Tutorials (PDF compilation, December 2004)*, 482 pages. Referenced as "NeHe 2004 PDF."

2.9.2 Supporting Technical References

- The Khronos Group, *OpenGL 3.3 Core Profile Specification*. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf
- The Khronos Group, *OpenGL Wiki — Legacy OpenGL*. https://www.khronos.org/opengl/wiki/Legacy_OpenGL
- PyOpenGL project, *PyOpenGL 3.x Documentation*. http://pyopengl.sourceforge.net/
- GLFW project, *GLFW 3.x Documentation*. https://www.glfw.org/docs/latest/
- FreeType project, *FreeType 2 Documentation*. https://freetype.org/freetype2/docs/
- ODE project, *Open Dynamics Engine Documentation*. https://www.ode.org/
- Shoemake, K. (1994), "Arcball Rotation Control," *Graphics Gems IV*, Academic Press. (ArcBall quaternion math reference for L48)

STAGE 3 — MISSION PACKAGE

3 Milestone Specification

3.1 Mission Objective

Produce a fully integrated NeHe OpenGL Intelligence Pack for the gsd-skill-creator dev branch: a YAML taxonomy registering all 48 lessons as skill nodes, working Python implementations for all 48 lessons (PyOpenGL + GLFW, cross-platform, pip-installable), automated tests for lessons 01–20, DACP bundle schemas, a deprecation map from legacy to modern OpenGL, and a documentation index enabling agent-driven concept lookup — delivered as a single droppable directory.

3.2 Architecture Overview

```
WAVE EXECUTION ARCHITECTURE
┌──────────────────────────────────────────────────────────────┐ │ WAVE 0:
Foundation (Sequential, Paula/Haiku) │ │ Shared schemas · lesson metadata
· dir scaffolding │ └──────────────────────────────────────────────────────────┘
│ ┌──────────────────┼──────────────────────────┐ ▼ ▼ ▼ ┌──────────────┐
┌──────────────────┐ ┌──────────────────┐ │ WAVE 1A │ │ WAVE 1B │
│ WAVE 1C │ │ Taxonomy  │ │ Code: L01-L24 │ │ Code: L25-L48 │ │
Intelligence │ │ Foundation + │ │ Advanced + │ │ Integration │ │
Intermediate │ │ Expert │ │ (Agnus/Opus) │ │ (Denise/Sonnet) │ │
(Denise/Sonnet)│ └──────────────┘ └──────────────────┘ └──────────────┘
└──────────────────┼──────────────────────────┘ │ CAPCOM GATE ▼
┌──────────────────────────────────────────────────────────────┐ │ WAVE 2:
Synthesis (Sequential, Agnus/Opus) │ │ Cross-module validation · DACP
schemas · Deprecation map │ │ Documentation index · Integration testing
│ └──────────────────────────┼──────────────────────────────────────────┘ │ CAPCOM
GATE ▼ ┌──────────────────────────────────────────────────────────┐
│ WAVE 3: Publication + Verification (Denise/Sonnet) │ │
README · skill-creator YAML · final test run · delivery │
└──────────────────────────────────────────────────────────┘
```

3.3 System Layers

1. **Schema Layer** (Wave 0) — Shared data structures: lesson metadata schema, DACP bundle schema, directory layout manifest
2. **Taxonomy Layer** (Wave 1A) — YAML skill taxonomy, JSON-LD concept DAG, deprecation map, intelligence index
3. **Implementation Layer** (Waves 1B, 1C) — 48 Python lesson files, organized by tier, with inline docs
4. **Test Layer** (Wave 1B/1C, finalized Wave 2) — pytest suite for L01–L20, visual spot-check docs for L21–L48
5. **Integration Layer** (Wave 2) — Cross-module linkage validation, DACP bundle generation, skill registration test

6. **Publication Layer** (Wave 3) — README, CHANGELOG, dev branch integration guide, final delivery

3.4 Deliverables

#	Deliverable	Acceptance Criteria	Spec
D1	Lesson taxonomy YAML	48 entries, valid schema, all fields populated	M1
D2	JSON-LD concept DAG	Valid JSON-LD, prerequisite edges correct	M1
D3	Deprecation map	15+ patterns mapped, severity classified	M6
D4	DACP bundle schema	Validates against 3-part bundle spec	M6
D5	Documentation index	30/30 concept queries return correct lesson	M6
D6	L01–L12 Python (×12)	Runs on Ubuntu 24, visual output correct	M2
D7	L13–L24 Python (×12)	Runs on Ubuntu 24, no dependency errors	M3
D8	L25–L36 Python (×12)	Runs on Ubuntu 24, stubs noted for L35	M4
D9	L37–L48 Python (×12)	Runs on Ubuntu 24, L47 stub noted	M5
D10	Test suite (L01–L20)	All tests pass in fresh environment	M2/M3
D11	skill-creator YAML	Installs via skill registration process	M6
D12	README + integration guide	Single-command drop-in to dev branch	M6

3.5 Component Breakdown

Component	Dependencies	Model	Est. Tokens	Notes
Lesson taxonomy YAML	lesson metadata schema	Haiku	4k	Structured, templated
JSON-LD concept DAG	taxonomy YAML	Sonnet	8k	Graph construction
Deprecation map	taxonomy, OpenGL spec	Opus	12k	Requires API judgment
DACP bundle schema	3-part bundle spec	Opus	8k	Schema design
Documentation index	taxonomy + DAG	Sonnet	6k	Index construction
L01–L05 Python	GLFW schema	Sonnet	15k	Foundation, 5 files
L06–L12 Python	texture schema, light schema	Sonnet	28k	Texture/light/blend
L13–L20 Python	font schema, particle schema	Sonnet	32k	Complex patterns
L21–L28 Python	ext schema, stencil schema	Sonnet	35k	Stencil/shadow heavy
L29–L36 Python	terrain schema, RTT schema	Sonnet	38k	Simulation patterns
L37–L48 Python	physics schema, VBO schema	Opus+Son	55k	Expert tier, quaternions
Test suite L01-L20	implementations GL stub	Sonnet	20k	pytest + framebuffer
skill-creator YAML	taxonomy, skill spec	Sonnet	6k	Registration format
README + guide	all deliverables	Sonnet	8k	Integration docs

3.6 Model Assignment Rationale

Model	Budget %	Assigned Tasks	Rationale
Opus	30%	Deprecation map design, DACP schema architecture, expert tier code (L37–L48), cross-module integration synthesis	Judgment-heavy: API evolution analysis, quaternion math, physics system design
Sonnet	60%	All foundation through advanced Python implementations (L01–L36), tests, taxonomy compilation, index construction, README	Structural implementation: high-volume, well-specified code generation
Haiku	10%	Lesson metadata schema, directory scaffolding, template stubs, test fixture generation	Fast scaffolding: templated, low-judgment work

4 Wave Execution Plan

4.1 Wave Summary

Wave	Name	Mode	Model	Output
0	Foundation	Sequentia	Haiku	Shared schemas, directory layout, lesson stubs
1A	Taxonomy Track	Parallel	Opus	YAML taxonomy, DAG, DACP schemas, deprecation map
1B	Foundation+Inte Code	Parallel	Sonnet	L01–L24 Python implementations + tests
1C	Advanced+Expe Code	Parallel	Sonnet+Opus	L25–L48 Python implementations + tests
GATE	CAPCOM Review	Sequentia	Human	Scope confirm, visual checks, test results review
2	Synthesis	Sequentia	Opus	Cross-module validation, index, integration
GATE	CAPCOM Review	Sequentia	Human	Final go/no-go before publication
3	Publication	Sequentia	Sonnet	README, skill YAML, final delivery

4.2 Wave 0 — Foundation

Task	Agent	Output
Create directory structure	Paula (Haiku)	`nehe-intelligence-pack/{lessons,taxonomy,tests,c`
Define lesson metadata schema	Paula (Haiku)	`schemas/lesson_metadata.yaml`
Define DACP bundle template	Paula (Haiku)	`schemas/dacp_bundle.yaml`
Generate lesson stub files	Paula (Haiku)	48 × `lessons/lessonNN/lessonNN.py` (stubs)
Generate test fixture templates	Paula (Haiku)	`tests/conftest.py`, test stubs

4.3 Wave 1 — Parallel Tracks (A, B, C)

4.3.1 Track A — Taxonomy and Intelligence

Task	Agent	Output
Build YAML taxonomy (48 entries)	CRAFT-ARCH (Opus)	`taxonomy/nehe_lessons.yaml`
Build JSON-LD concept DAG	CRAFT-ARCH (Opus)	`taxonomy/concept_dag.json`
Design DACP bundle schemas	ARCH (Opus)	`schemas/dacp_opengl.yaml`
Build deprecation map	ANALYST (Opus)	`taxonomy/deprecation_map.yaml`
Design documentation index	INTEG (Sonnet)	`docs/index.yaml`

4.3.2 Track B — Foundation & Intermediate Code (L01–L24)

Task	Agent	Output
Implement L01–L05 (Foundation I)	EXEC-1 (Sonnet)	5 × lesson py + tests
Implement L06–L12 (Foundation II)	EXEC-1 (Sonnet)	7 × lesson py + tests
Implement L13–L18 (Intermediate I)	EXEC-2 (Sonnet)	6 × lesson py + tests
Implement L19–L24 (Intermediate II)	EXEC-2 (Sonnet)	6 × lesson py + tests
Run test suite L01–L20	VERIFY (Sonnet)	Test results, defect log
Fix defects L01–L20	EXEC-1/2 (Sonnet)	Patched implementations

4.3.3 Track C — Advanced & Expert Code (L25–L48)

Task	Agent	Output
Implement L25–L30 (Advanced I)	EXEC-3 (Sonnet)	6 × lesson py
Implement L31–L36 (Advanced II)	EXEC-3 (Sonnet)	6 × lesson py
Implement L37–L42 (Expert I)	CRAFT-IMPL (Opus+Sonnet)	6 × lesson py + math docs
Implement L43–L48 (Expert II)	CRAFT-IMPL (Opus+Sonnet)	6 × lesson py + math docs
Visual spot-check L21–L48	VERIFY (Sonnet)	Screenshot docs + notes

4.4 Cache Optimization Strategy

- **Shared lesson schema:** Computed once in Wave 0, cached for all 48 implementation agents. Each agent loads the schema at start, reducing redundant schema reasoning.
- **GLFW boilerplate:** L01 establishes the window + context pattern. All subsequent lessons inherit this via a shared `lessons/common/glfw_window.py` module — agents reference it, not re-implement it.
- **Texture loading utility:** L06 establishes the Pillow-based texture loader. All texture-using lessons (L07–L48) import from `lessons/common/texture.py`.
- **Taxonomy preload:** Track A output is cached at 5-minute TTL and preloaded into Track B/C context to enable DACP bundle population without re-computing metadata.
- **Test fixtures:** Shared framebuffer comparison utility in `tests/conftest.py` loaded once per test session.

4.5 Token Budget

Wave/Track	Opus (k)	Sonnet (k)	Haiku (k)	Notes
Wave 0	0	0	20	Scaffolding only
Wave 1A	38	6	0	Taxonomy + schemas
Wave 1B	0	110	0	L01–L24 code + tests
Wave 1C	20	93	0	L25–L48 code
Wave 2	25	20	0	Synthesis + integration
Wave 3	0	22	5	Publication
Total	**83k**	**251k**	**25k**	≈359k total

4.6 Dependency Graph

```
DEPENDENCY GRAPH
[W0: Schemas] | ┌──────┴──────────────────────────────────────────────────┐
| | | [W1A: Taxonomy] [W1B: Code L01-24] [W1C: Code L25-48]
| | | | [CAPCOM GATE: Visual check + test pass] | | |
└──────────────────────┴──────────────────────────────┘ | [W2: Synthesis +
Integration] DACP bundles validated Taxonomy cross-linked Documentation
index built | [CAPCOM GATE: Final review] | [W3: Publication]
skill-creator YAML README + guide Final delivery
```

5 Test Plan

5.1 Test Categories

Category	Count	Priority	Action on Fail
Safety-critical	4	P0	BLOCK delivery
Core functionality	48	P1	Fix before Wave 3
Integration	12	P1	Fix before Wave 3
Regression	20	P2	Document & defer
Edge cases	15	P3	Document
Visual spot-checks	28	P2	Screenshot doc

5.2 Safety-Critical Tests (BLOCK delivery if fail)

ID	Verifies	Expected Result
SC-01	No undeclared dependencies	`pip install -r requirements.txt` in fresh venv succeeds; all imports resolve
SC-02	No Windows-only code in lessons 01–34	`grep -r "win32|glaux|wgl"` returns zero matches outside explicitly documented stubs
SC-03	L45 uses only OpenGL 3.3 core profile	No glBegin/glEnd, no fixed-function calls in lesson45.py; validated by code parser
SC-04	All lesson files include MIT attribution	`grep -l "Jeff Molofee"` returns 48 files; validates copyright compliance

5.3 Core Functionality Tests

Test ID	Lesson	Test Method
CF-01 to CF-12	L01–L12	Headless framebuffer render (pytest-gl); compare against reference PNG within 2% RMSE
CF-13 to CF-20	L13–L20	Import succeeds; init sequence completes; first frame renders without GL error
CF-21	L45 VBO	Buffer binding, data upload, draw call all succeed; GL_NO_ERROR confirmed
CF-22	L48 ArcBall	Unit tests on map_to_sphere() and compute_rotation(); quaternion norm = 1.0 within 1e-6
CF-23	L43 FreeType	Glyph atlas generates 128 characters; no GL errors on atlas upload
CF-24	L39–L40 Physics	Spring-mass and Verlet systems advance 1000 steps without NaN/Inf

5.4 Verification Matrix

SC#	Success Criterion	Test IDs	Status
1	All 48 lessons in taxonomy YAML	CF-T1 (schema validator)	Pre-Wave 1A
2	All 48 Python implementations run	CF-01 to CF-20 + SC-01	Wave 1B/1C
3	Foundation visual correctness	CF-01 to CF-12	Wave 1B
4	DACP bundles validate	CF-T2 (DACP validator)	Wave 2
5	15+ legacy patterns in deprecation map	SC-02 + manual count	Wave 1A
6	Documentation index accuracy	CF-T3 (30 queries)	Wave 2
7	skill-creator YAML installs	CF-T4 (install test)	Wave 3
8	Tests pass in fresh environment	SC-01	Wave 3
9	Expert lessons include math docs	Manual review (CF-T5)	Wave 1C
10	ArcBall quaternion tests pass	CF-22	Wave 1C
11	L45 uses only 3.3 core profile	SC-03	Wave 1C
12	Single droppable directory	CF-T6 (structure check)	Wave 3

6 Mission Crew Manifest

Role	Agent	Model	Responsibility
FLIGHT	Orchestrator	Opus	Mission direction; wave go/no-go; scope arbitration
PLAN	Planner	Opus	Wave decomposition; dependency ordering; cache strategy
TOPO	Topology Manager	Sonnet	Agent team structure; module reassignment if needed
ARCH	Architecture	Opus	DACP schema design; deprecation map; taxonomy structure
SCOUT	Research Agent	Sonnet	Upstream source validation; PyOpenGL API research
EXEC-1	Foundation Executor	Sonnet	L01–L12 implementation
EXEC-2	Intermediate Executor	Sonnet	L13–L24 implementation
EXEC-3	Advanced Executor	Sonnet	L25–L36 implementation
CRAFT-IMPL	Expert Implementer	Opus+Sonr	L37–L48 implementation with math docs
CRAFT-ARCH	Taxonomy Architect	Opus	YAML taxonomy, JSON-LD DAG
ANALYST	Pattern Analyst	Opus	Cross-lesson pattern detection; deprecation analysis
VERIFY	Verification	Sonnet	Test execution; source validation; GL error checking
INTEG	Integration Agent	Sonnet	Cross-module linkage; index construction
SURGEON	Health Monitor	Sonnet	Code quality degradation detection
SECURE	Security	Sonnet	License compliance; copyright header validation
CAPCOM	HITL Interface	Human	Wave boundary approvals; visual checks
RETRO	Retrospective	Sonnet	Post-wave summaries; defect pattern analysis
LOG	Chronicle	Haiku	Commit messages; milestone summaries

7 Execution Summary

Metric	Value
Total lessons covered	48
Platform ports in archive	43
Target implementation language	Python (PyOpenGL + GLFW)
Activation profile	Fleet (17 roles)
Estimated context windows	8–12
Estimated sessions	3 (Wave 0+1 / Wave 2 / Wave 3)
Total token estimate	≈359k
Opus %	≈23% (83k)
Sonnet %	≈70% (251k)
Haiku %	≈7% (25k)
CAPCOM gates	2 (post Wave 1; post Wave 2)
Safety-critical tests (BLOCK)	4
Core functionality tests	24
Total deliverables	12
Estimated Python lines of code	6,000–9,000 (48 lessons × 125–190 LoC avg)
Dev branch integration	Single directory drop-in
License	MIT (Jeff Molofee / NeHe archive)

"The Amiga rendered what IBM machines could only approximate — not because it had more power, but because it had the right architecture. Agnus knew DMA. Denise knew color. Paula knew rhythm. Three chips, each doing one thing perfectly, composing effects that seemed to exceed what the underlying hardware should allow."

The NeHe corpus is the OpenGL equivalent of that moment: 48 lessons, each doing one thing perfectly, composing a complete education in how to build worlds from vertices and pixels. This intelligence pack doesn't just archive that knowledge — it wires it into skill-creator's nervous system, so the lessons can be recalled, composed, and executed at machine speed, exactly when they're needed, in exactly the form they're needed in.

The spaces between the lessons — the prerequisite edges in the DAG, the shared patterns that emerge across tiers, the moment a particle system and a physics simulator start to look like the same pattern at different scales — that's where the intelligence lives.